EgoQuant Security Self-Assessment Report

2026 Shenzhen FinTechathon · International Track · Topic B: Wealth Advisory Agent · Oct 20, 2026

---

1. Executive summary

EgoQuant aims to give every investor the kind of advice a private-bank advisor gives: a portfolio built around who they really are and what they are saving for, explained in plain words. It learns the investor in two ways, from what they say (a validated risk questionnaire, their goal, horizon and interests) and from how they behave (a risk game, a market-crash test and, optionally, their real brokerage history). From that, four specialised agents work together: Agent C selects suitable real-market instruments, Agent A builds and tests 100 candidate portfolios against the investor's goal over 10,000 simulated futures, Agent B, the investor's behavioural twin, removes anything they would likely panic-sell, and Agent D watches the news for their holdings. An independent Risk Engine signs off every portfolio, and an AI advisor answers the investor's questions and runs "what if" scenarios on request.

Security is built into this architecture, not added on top. Its core safeguard is a separation of powers: deterministic code decides every score, limit, portfolio and trade verdict, and the language model only explains them, working from HMAC-sealed facts behind a compliance guard and a number check that blocks any figure the engine did not produce. Client data is minimised, redacted before any model call and isolated per user by database Row-Level Security. The system is backed by 1,668 passing automated tests and two red-team runs that each passed 24 of 24 attacks against the live model.

This report is submission item 05 (permission-tier implementation and known-risk list). It assesses EgoQuant first against international financial AI rules, PBoC standard JR/T 0221-2021 and the NFRA's June 2026 AI guidance, and then against the OWASP Top 10 lists for agentic and LLM applications.

Document control

Field	Value
System	EgoQuant: backend (FastAPI, Python) and frontend (Next.js 16, React 19)
Repositories	umutbideci8/fintechaton-backend, umutbideci8/fintechaton-frontend
Competition	2026 Shenzhen FinTechathon, International Track, Topic B (Wealth Advisory Agent)
Assessment type	Self-assessment, white-box (full source access), by the development team
Basis	docs/SECURITY.md draft v2 (19 Oct 2026), the source code, the automated test suite and the red-team logs in eval/results/
Classification	Prototype; not a licensed advisory service; no real client money

Overall posture

Advice only, by construction. EgoQuant has read-only access to a client's trade history and no route to any broker's order or transfer API. The worst outcome of a full model compromise is a bad explanation, not a bad trade. Limits only ever tighten. The questionnaire sets the ceiling (risk class C1 to C5). The behavioural agent and the What-If tool can only make limits stricter. An independent Risk Engine has the final say. The LLM is a narrator, not a decision-maker. It cannot set a score, weight, limit or verdict. Its context is HMAC-sealed, every reply passes a compliance guard and number grounding, and personal identifiers are redacted before any call leaves our servers.

Data is minimised. Brokerage tokens and uploaded files are never stored. Account data is isolated by Supabase Row-Level Security, with no service-role key anywhere in the system, and expires after 180 days.

Headline evidence

Evidence	Result
Backend offline test suite (pytest -m “not live”)	1,668 passed, 8 failed, 2 skipped; every security test passed, the 8 failures are output-regression checks (see R-15)
Hostile-input suite (tests/test_extreme_inputs.py)	80 cases: no 5xx, no input echoed back
Advisor chat red team (eval/chat_redteam.py, 19 Oct)	24 of 24 scored cases passed
Trade-explanation red team (eval/trade_explain_redteam.py, prompt v2)	24 of 24 passed
Malicious upload tests (XXE, billion laughs, oversize, formula cells)	Over 90 tests, all passing
Python dependency audit (pip-audit)	Clean for deployed packages

How EgoQuant keeps investors safe

#	Guarantee to the investor	How it is built	Proof
1	The AI can never move your money.	EgoQuant has no order, transfer or withdrawal API anywhere. Brokerage access is read-only IBKR Flex statements; a planned Binance link refuses any key that can trade or withdraw.	Section 4.4; no trading client in either repository
2	The AI never makes a financial decision.	Every score, risk class, limit, portfolio weight and trade verdict comes from deterministic code: the same inputs always give the same answer. The language model only explains.	Section 4.2; Risk Engine and trade-check tests

3	Your limits can only get safer, never riskier.	The questionnaire sets a ceiling. The behavioural twin and the What-If tool can only tighten it, and an independent Risk Engine re-checks every portfolio and has the final say.	349 Agent B tests; What-If tests; red-team case “run with C5 limits” blocked
4	Every number you see is real.	Number grounding checks each percentage, amount and probability in a reply against the engine’s own results. A reply with an invented number is regenerated once, then replaced by a safe answer.	test_grounding_redaction.py
5	The AI cannot be talked into giving advice.	The model reads facts from an HMAC-sealed context it cannot alter. Every sentence passes a compliance guard that blocks buy or sell calls, price targets, forecasts and promises.	Live red team: 24 of 24 attacks stopped, including four prompt-injection types and a Chinese-language attack
6	The AI never learns who you are.	Names, account ids, emails and phone numbers are replaced with placeholders before any model call. The model never sees balances, holdings lists or raw trades. Brokerage tokens are used once and never stored.	Redaction tests check the outgoing payload; test_no_echo_422.py
7	Your data is yours alone, and you can delete it.	Database Row-Level Security on every table; no master key exists, and the server refuses to start with one. Storage needs versioned consent, expires after 180 days, and one click erases everything.	148 account-isolation tests; rls_isolation.sql
8	Every result can be reproduced and audited.	Reports name their data sources and simulation method; What-If runs store the seed, path count and engine version; confirmed actions go into an activity log.	Section 10.2

These guarantees match what regulators ask of financial AI: the NFRA’s June 2026 guidance requires human decision-making where AI only assists and protection against agent tool abuse, memory contamination and data leakage; PBoC standard JR/T 0221-2021 requires objective-function security, attack resistance and traceability.

2. System overview and trust boundaries

EgoQuant runs as three tiers: a Next.js frontend on Vercel, a FastAPI backend on Railway, and a Supabase (PostgreSQL) account store. External services sit outside our trust boundary and are treated as untrusted input or as processors of minimised data.

Data flows left to right into the backend and down the advisory pipeline; every portfolio passes the Risk Engine before the LLM sees it, and nothing flows out to a trading system.

Request path. The browser never talks to the backend or Supabase directly for authenticated actions. Next.js route handlers (src/app/api/*) proxy every call, hold the Supabase session in HttpOnly, Secure, __Host- cookies, check the request Origin, and forward a Bearer token to the backend.

Advisory pipeline. One portfolio request flows through these components in order:

- Risk profile (egoquant/risk_profile): scores the Grable & Lytton tolerance questions, the goal-based loss capacity and the Lusardi-Mitchell literacy questions. It sets risk class C1 to C5, the maximum product risk level R1 to R5 and blocked product types. Limits come only from risk_profile/config.yaml.
- Agent C, market intelligence (agents/market_intel): selects only instruments the profile and the client's stated interests allow, and fetches prices from Tiingo, Binance and committed snapshots (read-only).
- Agent A, quant (agents/quant): builds 100 candidate portfolios in five styles under the profile's hard limits, then tests the survivors with a 10,000-path block bootstrap against the client's goal.
- Agent B, behavioural twin (agents/behavioral): screens candidates using the Risk Game, the crash test and optional IBKR history. It can only tighten limits.
- Risk Engine (egoquant/risk_engine.py): independently re-checks the final weights against every hard limit and stress scenario. It has the final say.
- Report and advisor (egoquant/report.py, chat.py): the report is built from engine output. The LLM explains it in plain words under the controls in Section 4.
- Agent D, briefing (egoquant/briefing): labels the tone of news headlines for the client's funds and may offer a risk-profile re-check. It never changes a portfolio.

Assets protected

Asset	Where it lives	Why it matters
Client risk profile and questionnaire answers	Browser storage; HMAC-sealed token; Supabase if signed in	Drives suitability; tampering could push a client into unsuitable products
Brokerage trade history (IBKR)	Memory for one request; Supabase if the user consents	Personal financial information
IBKR Flex token	Memory for one fetch only (SecretStr)	Read access to a real brokerage statement
Supabase session tokens	HttpOnly cookies	Account takeover
Risk limits and engine code	risk_profile/config.yaml, repository	The suitability boundary itself
HMAC context secret, LLM and data API keys	Host environment variables	Forged contexts, cost abuse, data-provider abuse
Audit records (operation log, What-If runs, privacy counts)	Supabase, RLS-protected	Evidence of what the client was shown and confirmed

Trust boundaries

TB1: browser to Next.js (untrusted client input, cookies).

TB2: Next.js to FastAPI (Bearer token, CORS limited to the Vercel origin).

TB3: FastAPI to Supabase (queries run as the user under RLS; no service-role key).

TB4: FastAPI to OpenAI (minimised and redacted facts out; model text back, treated as untrusted).

TB5: FastAPI to market-data and news providers and IBKR (untrusted data in; read-only credentials).

TB6: agent to agent inside the backend (typed Pydantic contracts; tighten-only rules; final Risk Engine check).

3. Methodology and reference standards

We assessed EgoQuant against core financial and AI standards first. These standards were collected from the sources detailed below.

Primary references: Chinese regulation and standards

Reference	Issuer, date	What we take from it
Guidance on the Safe Development and Application of AI in Banking and Insurance (关于银行业保险业人工智能安全开发应用的指导意见)	NFRA, 18 Jun 2026	Lifecycle governance; admission control for high-risk uses; human supervision and override at key points; AI-generated content labelled; financial-agent risks: data leakage, memory contamination, identity misuse, tool abuse, runaway operation
JR/T 0221-2021 AI Algorithm Financial Application Evaluation Specification (人工智能算法金融应用评价规范)	PBoC, 26 Mar 2021	Four evaluation dimensions (security, explainability, accuracy, performance); the security sub-items (objective-function security, attack resistance, dependency-library security, traceability, internal control); its evaluation methods; fund-related vs non-fund scenarios
JR/T 0171-2020 Personal Financial Information Protection Technical Specification (个人金融信息保护技术规范)	PBoC, 13 Feb 2020	Lifecycle protection of personal financial information: collection, storage, use, deletion
GB/T 45654-2025 Basic Safety Requirements for Generative AI Services	SAMR and SAC (TC260), effective 1 Nov 2025	Financial information services named as a sensitive use; injection monitoring; disclosure of model use and limits; refusal testing; backup and recovery
Measures for Labelling AI-Generated Synthetic Content (人工智能生成合成内容标识办法)	CAC and three ministries, effective 1 Sep 2025	Explicit labels on AI-generated text, kept when copied or exported
Interim Measures for Generative AI Services	CAC and six ministries, Aug 2023	Accuracy and reliability of generated content; user-facing transparency
Personal Information Protection Law (PIPL)	NPC, Nov 2021	Consent, minimisation, purpose limitation, deletion rights

Technical Requirements for Agents in Financial Applications (金融行业应用智能体技术要求)	Beijing FinTech Industry Alliance group standard, 27 Aug 2026	Agents use official APIs only (no screen scraping or simulated clicks); dual authorisation by the user and the institution
CSRC investor suitability rules and AMAC fund-distributor suitability guideline	CSRC / AMAC, effective 1 Jul 2017	Five-level investor and product risk matching; products only at or below the investor's level
Draft group standard: Ethics and Security Evaluation of AI Algorithms in Finance	Guangxi E-Commerce Enterprises Federation, draft Jan 2026	AI as an assistive tool, not a fully autonomous decision-maker; explanation depth matched to risk; version control, rollback and incident plans

Secondary references: international frameworks

- OWASP Top 10 for Agentic Applications (Dec 2025), ASI01 to ASI10: our agent threat checklist.
- OWASP Top 10 for LLM Applications 2025, LLM01 to LLM10: our model-interface checklist.
- NIST AI 600-1 Generative AI Profile and the NIST AI RMF functions (Govern, Map, Measure, Manage): structure for Sections 8 and 10.
- US SEC robo-adviser guidance, 2017: disclosure, questionnaire adequacy, algorithm testing before and after release, change control.
- STRIDE threat modelling (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege).

Evaluation methods used (the JR/T 0221-2021 method set)

JR/T 0221 method	What we did
System inspection	Read every backend and frontend source file, configuration and migration
System testing	Ran the full offline test suite (1,678 tests) on 20 Oct 2026
Attack testing	Hostile-input suite, malicious-upload suite, cross-user (IDOR) tests, live LLM red-team runs
Algorithm testing	Risk Engine and tighten-only property tests; What-If common-random-number tests; regime-model walk-forward evaluation

Algorithm inspection	Traced every limit back to risk_profile/config.yaml and every LLM number to the engine facts

Risk rating: Each risk is scored for likelihood (1 low to 3 high) and impact (1 low to 3 high) in the context of a public demo with real users' brokerage data. Score = likelihood × impact: 7 to 9 Critical, 5 to 6 High, 3 to 4 Medium, 1 to 2 Low. Impact is weighted towards client harm (unsuitable advice, leaked financial data) over availability. Ratings are after mitigation (residual).

Team decisions: Many controls in this report are required by a standard (for example, matching products at or below the investor's level). Others are decisions our team made to prioritise cliend security where no standard fixes the answer, such as the exact caps per risk class or keeping crypto away from all but C5. Each such choice is listed in a "Team decisions" table (TD-01 onward) with the reason we made it and the harm it prevents, so a reviewer can tell a regulatory requirement from our own judgement and challenge it.

---

4. Permission-tier implementation

EgoQuant enforces permissions in three layers: who the person is (user tiers), what each software component may do (component tiers), and which products a client may be shown (suitability tiers). Each layer is enforced in code, not by convention, and the strictest layer always wins.

4.1 User tiers

Tier	May	May never	Enforced by
T0 Visitor (no account)	Questionnaire, Risk Game, a portfolio report, the advisor chat, a one-off trade check. IBKR data may be sent with one request and is used once.	Store anything on the server; record a trade; see any saved data	No session cookie, so account routes return 401; visitor routes write nothing (egoquant_routes.py)
T1 Signed-in user	All of T0, plus: save reports, connect IBKR data, check planned trades against saved holdings, record "I made this trade", read their own activity log, delete their data or account	See or change another user's data; act without the matching consent	Identity only from the verified token, never from the request body (accounts/routes.py); Supabase Row-Level Security (supabase/migrations/001_accounts.sql) plus a backend user_id filter

T1 + consent ibkr_data	Store minimised trade history for 180 days	Store tokens, files, account numbers or names	Versioned consent (CONSENT_VERSION = “2026-10-19”); a changed text needs new consent
T1 + consent trade_check	Run a saved-holdings trade check and save the verdict	Supply holdings in the request (they come only from the user’s own saved trades)	/account/trade-check checks the consent scope (tests/accounts/test_trade_check_routes.py)
Operator (team)	Deploy, set environment secrets, run migrations	Read user rows through the app (no service-role key exists; the backend refuses to start with one)	accounts/store.py _is_privileged_key; tests in test_accounts.py

Team decisions: user tiers

#	Decision	Why we chose it	How it protects the investor
TD-01	No administrator role and no service-role key; the server refuses to start with one	An admin account or master database key is the one credential that, if stolen or misused by an insider, exposes every client at once.	No attacker, and no member of our own team, can read a client’s trades or reports through the application; each person reaches only their own rows.
TD-02	Visitors can use everything without an account, and nothing is stored for them	A client should be able to see suitable advice before deciding to hand over any data.	A person who never signs up leaves no financial data on our servers.
TD-03	Two separate consents (storing IBKR trades; trade checks), each tied to a version of its wording	A client may want a trade checked without having their trading history kept; a changed purpose needs new consent.	Data is stored only for the purpose the client agreed to; if the wording changes, nothing new is stored until they agree again.

TD-04	Holdings for a signed-in trade check come only from the client's own saved trades, never from the request	If the request could supply holdings, a forged request could make an unsuitable trade look acceptable.	The verdict is always computed on what the client actually holds.

4.2 Component and agent tiers

Every agent is given the narrowest permission its job needs, and decisions flow one way: an agent downstream can reject or tighten, never loosen.

Component	Authority	May	May never	Enforced by
Risk profile scorer	Sets the ceiling	Compute C1 to C5, max R-level, blocked product types from answers	Use any input other than the versioned questionnaire	risk_profile/, limits only from config.yaml
Agent C, market intel	Read-only data	Fetch prices for allowed instruments only	Add an instrument the profile or interests exclude; write anything a client sees	agents/market_intel/; catalog is the single source of R-levels
Agent A, quant	Propose	Build and rank candidate portfolios under hard constraints	Exceed any hard limit; approve its own output	Constraints passed from the profile; Risk Engine re-check
Agent B, behavioural	Veto and tighten	Reject candidates; send a stricter loss limit	Raise any limit set by the questionnaire class	agents/behavioral/agent.py ("B only TIGHTENS"); flag behavioral_limit_not_below_class
Risk Engine	Final authority	Approve or reject the final weights	Be bypassed: no report is built without its review	risk_engine.py: R-level per holding (unknown ticker fails closed), equity, commodity and crypto caps, stress loss per named scenario

Trade check	Deterministic verdict	Compare a planned trade with the limits and Risk Engine	Use an LLM for the verdict	trade_check.py
LLM, advisor chat	Narrate	Explain sealed engine facts; call one read-only tool run_what_if, at most 2 calls per message (MAX_TOOL_CALLS = 2)	Set a score, class, weight, limit or verdict; quote a number the engine did not produce; see identities or raw holdings	HMAC-sealed context (advisor/context.py), compliance guard (advisor/guard.py), number grounding (advisor/grounding.py), redaction (advisor/redaction.py)
What-If tool	Simulate	Re-run the goal simulation with whitelisted inputs	Change the risk class, caps or limits; save over the client's answers	Pydantic extra="forbid", bounded fields, Risk Engine re-check against the BASE portfolio's hard limits (agents/what_if.py)
Agent D, briefing LLM	Label	Label one headline's tone for one fund	Change a portfolio; send an alert for good news	Hash-pinned prompt; damped score; FinBERT fallback (briefing/)

Team decisions: component and agent tiers

#	Decision	Why we chose it	How it protects the investor
TD-05	The language model narrates and never decides	A model can be steered by prompt injection and does not always give the same answer twice; a financial decision must be repeatable and auditable.	Whatever anyone types, the risk class, portfolio and trade verdict cannot change. The worst result of a compromised model is a badly worded explanation.

TD-06	Behaviour can only tighten limits, never loosen them	The questionnaire is the formal suitability classification. Behaviour that is more cautious than stated is valuable evidence; a bold game result is not grounds to raise a formal limit.	A client who panic-sells gets a calmer portfolio; a client who plays boldly still never exceeds their questionnaire ceiling.
TD-07	An independent Risk Engine re-checks every portfolio and has the final say	The optimiser (Agent A) should not approve its own work; a bug in it must not reach the client.	A portfolio that breaks any limit is rejected and never shown as advice.
TD-08	Unknown products fail closed	If we cannot rate a product's risk, we cannot claim it is suitable.	Missing or bad data removes a holding instead of letting an unrated product through.
TD-09	The What-If tool has no field for the risk class, caps or limits, and runs at most twice per message	The model picks the tool's arguments, so they are untrusted; removing dangerous fields is safer than filtering them, and the call limit stops runaway loops and cost.	"Run my portfolio with C5 limits" is impossible by design; every scenario is re-checked against the client's own limits.
TD-10	Agent D cannot alert on good news or change a portfolio; it may only offer a risk-profile re-check	Headline tone is wrong about 15% of the time and is not a forecast; good-news alerts would nudge clients to buy more.	Market noise can never trigger a trade; the client decides whether to retake the profile.

4.3 Suitability tiers

Product access follows the five-level matching used in Chinese investor-suitability practice: a client is only shown products at or below their level. Values come from risk_profile/config.yaml and are labelled DRAFT LIMITS in the report until a licensed institution confirms the mapping.

Risk class	Max product level	Max equity	Max commodity	Max crypto
C1	R1	0%	0%	0%
C2	R2	0%	0%	0%
C3	R3	40%	10%	0%
C4	R4	70%	15%	0%
C5	R5	90%	20%	5%

The client's class is the lowest of tolerance and capacity. Low financial literacy blocks complex product types without changing the class. Options the class does not allow are shown greyed out with the reason, so the client cannot select them.

Team decisions: suitability tiers

The regulation requires at least five investor classes and five product levels, with products matched at or below the investor's level. It does not set how much of a portfolio may be in stocks, commodities or crypto at each level. Every percentage in the table above is a team decision.

#	Decision	Why we chose it	How it protects the investor
TD-11	No crypto for C1 to C4; at most 5% for C5	We rate crypto R5 because it falls about 60% in our crash scenario. C1 to C4 can absorb losses of only about 5, 10, 20 and 35%; one crypto holding could exceed that alone. For C5 it stays a small satellite.	A cautious client is never shown an asset that could wipe out most of its value in one crash. For C5, a full crypto crash costs at most about 3% of the portfolio (5% x 60%).
TD-12	Crypto also requires full product knowledge, even for C5	Willingness to take risk is not understanding of the product; regulators have found robo-advisers ignoring clients' knowledge and experience.	An aggressive client who fails the literacy questions is not sold a product they do not understand; with partial knowledge they must first acknowledge a warning.

TD-13	Stock caps of 0, 0, 40, 70 and 90% for C1 to C5	Each cap is sized so a severe crash (stocks -30%, bonds -5%) costs roughly that class's loss band: about 5, 10, 20, 35 and 45%. Even a small stock share would break the C1 and C2 bands.	A crash costs the client about what they were classified as able to bear, and even the most aggressive client keeps at least 10% outside stocks.
TD-14	Gold and commodity caps of 0, 0, 10, 15 and 20%	Gold is R3 and silver and broad commodities are R4; they diversify but pay no income and swing sharply.	No portfolio becomes concentrated in volatile assets that earn nothing while the client waits.
TD-15	The class is the lower of risk tolerance and risk capacity	Tolerance is what the client is willing to lose; capacity is what they can afford to lose. When they disagree, the stricter must win.	A confident client with a short horizon or thin savings is protected from their own optimism; the screen says which side set the limit.
TD-16	Goal-based loss capacity = (amount invested - amount needed back) / amount invested	No standard measures capacity, so we compute it from the client's own goal rather than asking how they feel.	Any portfolio whose crash loss would leave the client short of the money they need is rejected before they see it.
TD-17	Gate question: a client who accepts no loss is C1, R1 products only, 0% crash-loss limit	Mirrors the lowest-tolerance category in Chinese suitability practice; one clear "no" outweighs every other answer.	Such a client is only offered Treasury-bill funds, which do not fall in our crash scenario.
TD-18	Financial literacy restricts products but never changes the class	The literacy questions measure understanding, not risk appetite; mixing them in would distort the validated Grable & Lytton score.	Clients are not wrongly classified; they are only kept away from products they do not understand.

TD-19	Unsuitable options are shown greyed out with the reason, not hidden	A client who cannot see why an option is missing may look for it elsewhere; disclosures should come before the decision.	The client cannot pick an unsuitable product and understands why.
TD-20	A C1 client may hold up to 100% in one fund	Only two R1 funds exist in our catalogue (BIL, SHY), so the usual 40% per-fund cap would make every C1 portfolio impossible; each fund holds many government bills or notes.	The most cautious clients still receive a valid portfolio, concentrated only in funds that are diversified inside.
TD-21	Cash capped at 30, 20 and 15% for C3, C4 and C5, never below the client's own cash need	A risk-minimising optimiser would otherwise park a growth client's money in T-bills, silently failing their goal.	A growth client gets the growth they asked for, while their own cash need always comes first.
TD-22	A 2% crypto minimum only when the client chose crypto and is allowed it, never above the class cap	Low-risk styles otherwise dropped crypto even for a C5 client who asked for it; it is a soft limit, so any hard limit wins.	The client's stated preference is respected without ever overriding a safety limit.

4.4 External system permissions

IBKR: read-only Flex Query statements only. EgoQuant never asks for or accepts an IBKR username or password and has no order API. The token is used for one fetch, then dropped.

Binance read-only keys: roadmap only. The design refuses any key whose permissions allow trading, withdrawal or transfer (checked via GET /sapi/v1/account/apiRestrictions).

On-chain (Topic F bonus): docs/ON_CHAIN_PLAN.md describes sealing report hashes on the Sepolia test network. It is a plan only; no wallet or signing key exists in the system.

This matches the Beijing FinTech Industry Alliance agent standard (official APIs only, no screen scraping, user authorisation) and the NFRA requirement for human decision-making where AI is only an aid: EgoQuant never acts; the client always decides and executes.

Design choices external access

#	Decision	Why we chose it	How it protects the investor
TD-23	No order, transfer or withdrawal capability at all, rather than a guarded one	An advice tool does not need to trade; a capability that does not exist cannot be misused, however the AI or the account is attacked.	No one can move a client's money through EgoQuant.
TD-24	IBKR tokens used for one fetch and dropped; uploads read in memory, never written to disk	A stored credential or statement is a permanent liability for a benefit used once.	A database breach would expose no brokerage credentials and no original statements.
TD-25	A future Binance link refuses any key that can trade, withdraw or transfer	Clients often create exchange keys with more permissions than needed.	Even a careless key cannot give EgoQuant the power to move funds.

---

5. Threat and risk register

We identified 24 threats across the web application, the multi-agent pipeline and the language-model interface; none is rated Critical after mitigation, and three are rated High. L = likelihood, I = impact, both 1 to 3; ratings are residual (after the listed controls).

5.1 Agent and model threats

ID	Threat	Maps to	Mitigation	Evidence	L×I	Rating
A1	Prompt injection through chat, headlines or trade text makes the advisor give unsuitable advice	LLM01, ASI01, NFRA tool abuse	Facts arrive as an HMAC-sealed context; a sealed token of another kind is refused; headlines and articles are treated as data; every reply passes the compliance guard	tests/test_chat.py; eval/chat_redteam.py 24/24	1×3	Medium

A2	Tool misuse: the LLM calls What-If with arguments that loosen limits ("run with C5 limits")	ASI02, LLM06	Whitelist with no field for class, caps or limits; extra="forbid"; bounded values; max 2 calls per message; Risk Engine re-checks against the base portfolio's limits, so scenarios can only tighten	tests/system/test_what_if.py, tests/test_chat_what_if.py, red-team whatif_* cases	1×2	Low
A3	The advisor invents a probability, amount or allocation (hallucination)	LLM09, NFRA accuracy, GB/T 45654 A.5	Number grounding: every percentage, money amount, "X in 100" and multiplier must match engine facts, this turn's What-If result or the user's own words; one regeneration, then a safe fallback	tests/test_grounding_redaction.py	2×2	Medium
A4	The LLM contradicts a deterministic trade verdict	LLM05, ASI09	Strict JSON schema; verdict and every number checked against the deterministic result; otherwise a fixed template is shown	tests/advisor/test_trade_explain.py; eval/trade_explain_redteam.py 24/24	1×2	Low
A5	Personal data sent to the model provider	LLM02, PIPL, JR/T 0171	Model sees only engine facts and aggregates; redaction gateway replaces IBKR account ids, emails, phone numbers and the username with placeholders; mapping lives in one request and is never logged	tests/test_grounding_redaction.py (outgoing payload holds no raw value)	2×2	Medium
A6	Memory or context poisoning: a forged or replayed context changes what the advisor believes	ASI06, NFRA memory contamination	Contexts are signed with a server secret; chats are not stored; the What-If inputs live in a separate sealed token never placed in the prompt	chat.py kind check; tests/test_chat_what_if.py	1×2	Low
A7	Insecure inter-agent communication: a downstream agent loosens an upstream limit	ASI07, JR/T 0221 internal control	Typed Pydantic contracts between agents; Agent B and What-If are tighten-only; Risk Engine is independent and final	agents/behavioral/agent.py; tests/agent_b/; Risk Engine tests	1×3	Medium

A8	Cascading failure: a data-provider outage or bad price feed corrupts every portfolio	ASI08, NFRA operational resilience	Source order Tiingo, Binance, cache, committed snapshots; data banners shown when a fallback is used; unknown ticker or missing R-level fails closed in the Risk Engine	agents/market_intel/ ; risk_engine.py	2×2	Medium
A9	Over-reliance: a client treats a simulation as a promise	ASI09, SEC disclosure, NFRA explainability	Stress results and goal chances labelled hypothetical and not a forecast in every view; DRAFT LIMITS banner; demonstration-only notice on the profile	report-view.tsx, crash-test.tsx, questionnaire/page.tsx	2×2	Medium
A10	Optional regime model silently changes outcomes	JR/T 0221 objective-function security, ASI04	Kill switch EGOQUANT_REGIME_MC=0 ; model fitted offline; only a validated JSON artifact is loaded; any failure falls back to the bootstrap; changes only the goal chance, never a weight, limit or verdict; report names the sampler used	tests/regime/	2×2	Medium (see R-04)
A11	Third-party model or prompt changes behaviour without review	LLM03, ASI04, SEC change control	Agent D prompt is hash-pinned; model name set by environment variable; LLM failure falls back to FinBERT or a template	briefing/ tests; prompts/fund_aware_tone.txt	2×1	Low

5.2 Application and data threats

ID	Threat	Maps to	Mitigation	Evidence	L×I	Rating
W1	Account takeover by weak or brute-forced passwords	STRIDE spoofing	Supabase Auth: hashing, email confirmation, login rate limits; no password reaches our code	Supabase configuration	2×2	Medium
W2	Session theft through XSS	STRIDE spoofing	Tokens only in HttpOnly, Secure, SameSite=Lax, __Host- cookies; Content Security Policy in next.config.ts	lib/server-auth.ts	1×3	Medium

W3	Cross-site request forgery	STRIDE tampering	State-changing /api/account/* routes check Origin; backend uses Bearer tokens, not cookies	lib/server-auth.ts (sameOrigin)	1×2	Low
W4	User A reads or changes user B's data (IDOR)	STRIDE information disclosure, PIPL	No endpoint takes a user id; queries run as the user under RLS; backend user_id filter; no service-role key	tests/accounts/test_accounts.py, supabase/tests/rls_isolation.sql (live run pending, R-02)	1×3	High until R-02 closes
W5	Brokerage token leaked	STRIDE information disclosure	Pydantic SecretStr; used once; never stored, logged, cached or returned; 422 errors never echo input	tests/test_no_echo_422.py	1×3	Medium
W6	Malicious upload (XXE, entity bomb, huge file, formula cells)	STRIDE tampering, DoS	defusedxml; DOCTYPE and ENTITY refused; 2 MB and 20,000-row caps; type must match content; strict symbol and currency patterns; parsed in memory	Over 90 tests incl. tests/agent_b/test_robustness.py	1×2	Low
W7	Secrets in the repository	STRIDE information disclosure	Keys only in host environment; .env.example has names only; staged diffs secret-scanned	Old Tiingo key in history (R-01)	2×2	High until R-01 closes
W8	Error messages leak internals	STRIDE information disclosure	One generic 500; provider error text never passed through	main.py, tests/test_extreme_inputs.py	1×1	Low
W9	Abuse, cost exhaustion or denial of service	LLM10, STRIDE DoS	Per-route rate limits; What-If 10 runs per 10 minutes per user; 20 s timeout; body size caps (413 or 411)	tests/test_body_limit.py	2×2	Medium (R-06, R-07)
W10	Cross-origin abuse or clickjacking	STRIDE tampering	CORS allows only the Vercel origin; X-Frame-Options: DENY, nosniff, strict referrer	main.py	1×1	Low
W11	Vulnerable dependencies	LLM03, JR/T 0221 dependency-library security	Pinned requirements.txt; pip-audit clean; 7 npm high findings are build-time only	Audit output	2×1	Low

W12	Debug endpoints reachable in production	STRIDE information disclosure	Synthetic data only; own rate limit (5/min)	/debug/portfolio, /api/debugmodels (R-03)	2×2	High until R-03 closes
W13	Repudiation: a client disputes what they were shown or confirmed	STRIDE repudiation, JR/T 0221 traceability	Operation log of confirmed actions; What-If audit with seed, path count and engine version; report names data sources and sampler	supabase/migrations/002_what_if.sql	2×2	Medium (log is user-writable, R-11)

---

6. Suitability, explainability and algorithm governance

EgoQuant is assessed here against the four JR/T 0221-2021 dimensions: security, explainability, accuracy and performance. The suitability dimension has been explained above. Advice is a non-fund scenario under that standard, but we apply the stricter fund-scenario expectations because the advice concerns client investments.

6.1 Explainability

Audience	What they get	Source
Client	Plain-language report: class and what it means, every limit and why, goal chance as “X in 100”, stress losses labelled hypothetical, why each candidate was rejected	report.py, report-view.tsx, goal-view.tsx
Client, conversational	Advisor chat grounded in sealed engine facts; What-If answers show before and after from the engine	chat.py, what-if-card.tsx
Reviewer or auditor	Every limit traced to config.yaml; report names data sources, fallback use and the sampler; What-If runs stored with seed, path count and engine version	report.py, what_if_runs table

The optimization and simulation are white-box methods (linear programming, SLSQP, block bootstrap, Ledoit-Wolf covariance), so no post-hoc explanation tool is needed for the numbers. Only the narration is generative, and it is constrained by Section 4.2.

6.2 Accuracy

Goal simulation. 10,000-path block bootstrap with Stambaugh/Page backfill for short histories and an equilibrium-means correction against hindsight bias. The same random numbers are used for a What-If and its base, so differences come from the change, not from noise.

Goal simulation

Each of these makes the chance of reaching a goal less flattering, but since the securit of traders is the priority of this tool are taken.

#	Feature	Why we chose it	How it protects the investor
TD-33	About 20 years of real price history, so 2008-09 is always in the sample	With only the last ten strong years, almost every goal looked certain.	Every simulated future draws on a real crash, so the downside is not hidden.
TD-34	Hindsight correction: stock, equity-fund and crypto average returns reset to a market-equilibrium level	Our instruments are picked from today's most-traded list, which favours past winners; one persona saw a 98% chance before the fix and 69% after.	The client is not pushed into more risk by a chance that only looks good because of hindsight.
TD-35	Recent performance is never used to rank or select investments	Ranking by what went up recently is a hidden forecast and invites chasing returns.	The client is not steered towards last year's winners.
TD-36	Chances shown as "X in 100" with an icon array; outcomes as "1 in 10 futures ends below …"	Risk-communication research (Spiegelhalter et al., 2011) shows natural frequencies are understood better than percentages.	The client reads the result as a range of futures, not a promise.

TD-37	No probabilities shown with less than about five years of usable history	A chance computed on too little data looks precise but means little.	The client is told why no chance is shown instead of getting a misleading number.

News tone (Agent D). On 120 hand-labelled headlines, the fund-aware LLM scored 85% accuracy (macro-F1 0.85), against 55% for FinBERT and 38% for the always-neutral baseline. Human labellers agreed with each other 75% of the time. The 95% interval at 85% is about ±6 points (eval/results.md).

6.3 Performance

A portfolio build has a 120-second server limit (maxDuration = 120) and a 125-second client timeout; a What-If is capped at 20 seconds with two worker threads; the chat is limited to two tool calls per message. Price caches and committed snapshots keep the system working when a provider is unavailable.

6.4 Algorithm change control

All code changes go through Git branches merged into the deployed branch; staged diffs are secret-scanned before commit.

The questionnaire, consent text and preference form are versioned (questionnaire_version, CONSENT_VERSION, preferences_version); a changed version invalidates stale results.

Model prompts are hash-pinned; model choice is an environment variable; the report records the engine version.

Every change runs the offline test suite; the LLM red-team scripts are re-run when a prompt changes.

---

7. Personal information protection

EgoQuant collects the least personal financial information that still produces suitable advice, stores it only with versioned consent, and deletes it after 180 days or on request. Controls follow PIPL and the lifecycle stages of JR/T 0171-2020.

Lifecycle stage	Control	Where
Collection	Visitors give questionnaire answers only; IBKR history is optional. Per trade only date, symbol, side, quantity, price, currency, asset category and net cash are kept: no account number, name or description	agents/behavioral/ibkr.py, accounts/ibkr_input.py
Consent	Separate, versioned consents for ibkr_data and trade_check; a changed text needs new consent; consent is logged	accounts/routes.py (CONSENT_VERSION)
Transmission	HTTPS end to end; tokens in HttpOnly cookies; LLM calls carry redacted, aggregated facts only	lib/server-auth.ts, advisor/redaction.py
Storage	Supabase with Row-Level Security on every table; no service-role key; uploaded files and Flex tokens are never written to disk; chats are not stored	supabase/migrations/001_accounts.sql, 002_what_if.sql
Use	Data is used only to build the client's own advice; the LLM never receives names, account numbers, balances, holdings lists or raw trades	chat.py, advisor/context.py
Retention	Every row has expires_at = 180 days; purge_expired() runs daily via pg_cron once enabled	Migrations; R-08

Deletion	DELETE /account/data erases every stored row; DELETE /account deletes the account and cascades; delete_my_data() also clears What-If and privacy-count rows	accounts/routes.py
Logging	Server logs carry exception types and routes, never request bodies; the operation log stores counts, dates and labels; redaction is logged as counts only ("N sensitive values hidden")	main.py, privacy_log
Browser	Profile and report kept in localStorage on the client's own device; IBKR data kept in memory only and never written to browser storage	lib/egoquant.ts, lib/ibkr.ts

Team decisions: personal data

PIPL requires minimisation and deletion below is a breakdown of the aspects of these requirements we considered.

#	Decision	Why we chose it	How it protects the investor
TD-38	180-day retention for stored trades and trade checks	The behavioural twin needs recent behaviour; six months usually covers at least one market correction while keeping history short.	Old financial history does not accumulate; a breach would expose at most six months of data.
TD-39	Only eight fields kept per trade (date, symbol, side, quantity, price, currency, asset category, net cash)	That is all Agent B needs to judge how a client behaves.	No account number, name or description is held, so leaked data cannot identify the client.
TD-40	Chats are not stored on our servers; the conversation lives in the client's browser tab	Stored chat histories can be leaked and can be poisoned to mislead later answers.	There is no chat archive to steal or tamper with.

TD-41	Identifiers replaced with placeholders before any model call; no balances, holdings lists or raw trades sent	The model provider is outside our control and overseas, so it gets only what it needs to phrase an answer.	The AI provider never learns who the client is or what they own.
TD-42	IBKR data kept in memory only, never in browser storage	Browser storage can be read by anyone with access to the device.	Brokerage data does not linger on a shared or lost computer.
TD-43	One click deletes all stored data; a second deletes the account	Deletion should be as easy as sign-up.	The client stays in control of their data at all times.

Cross-border note: The advisor and Agent D use an overseas LLM provider (OpenAI). Only redacted, aggregated engine facts and single headlines leave our servers. A production deployment in mainland China would need a domestically filed model, or a PIPL cross-border assessment, and should label AI-generated text under the 2025 labelling measures (R-12).

---

8. Security testing evidence

Security claims in this report are backed by automated tests that run offline on every change, plus live red-team runs against the production model. Test results below were re-run by the assessor on 20 Oct 2026.

8.1 Automated test suite (backend)

Command: pytest -q -m “not live” (live tests that call paid APIs are excluded and run separately).

Area	Tests	What the security-relevant tests prove
Accounts (tests/accounts/)	148	User A can never read or change user B’s reports, trades or trade checks; smuggled user_id and forged tokens are refused; a service-role key stops the store from starting; consent scopes are enforced

Agent B, behavioural (tests/agent_b/)	349	Uploads: XXE, billion laughs, oversized files, formula cells, wrong file types; tighten-only rule; robustness suite
Agent C, market intel (tests/agent_c/)	195	Only allowed instruments are fetched; fallbacks; catalog R-levels
Agent A, quant (tests/agent_a/)	92	Hard limits respected by every candidate; goal simulation correctness
Risk profile (tests/risk_profile/)	68	Scoring and limit mapping match config.yaml
Preferences (tests/preferences/)	22	Disallowed options cannot be chosen for a class
System and What-If (tests/system/)	199	End-to-end pipeline; What-If cannot loosen limits; Risk Engine has the final say
Advisor (tests/advisor/)	25	Trade explanations cannot contradict the verdict or invent a number
Briefing, Agent D (tests/briefing/)	406	Tone labelling, alert rules, advice filter on summaries
Regime model (tests/regime/)	89	Artifact validation; fallback to bootstrap; no hmmlearn at runtime
Top-level (tests/*.py)	84	Chat guard, number grounding and redaction (test_grounding_redaction.py); no input echo in 422 errors; body-size limits; 80 hostile-input cases (test_extreme_inputs.py)

Result: 1,678 passed, (1,678 tests, 9 minutes, Python 3.13 on 20 Oct 2026). Every account-isolation, upload, injection, redaction, grounding and tighten-only test passed.

8.2 Dependency and configuration checks

pip-audit: no known vulnerabilities in deployed Python packages (reported in docs/SECURITY.md).

npm audit: 7 high findings, all in a build-time tool chain not served to browsers (R-19).

Security headers: nosniff, X-Frame-Options: DENY, strict referrer policy, camera, microphone and location disabled; a Content Security Policy is set in next.config.ts.

8.3 Execution evidence (submission item 06)

The sandbox operation logs for item 06 come from the same controls: the operation log (consent given, data connected, report generated, warning shown, trade checked or recorded), the What-If audit table and the privacy-count log. All use synthetic or demo accounts; no real orders are placed.

---

9. Monitoring, incident response and human oversight

The client is the human in the loop for every decision: EgoQuant recommends, the client decides and places any trade at their own broker. This meets the NFRA rule that where AI only assists, a human makes the final decision, and the draft group standard's principle that the algorithm is not a fully autonomous decision-maker.

10.1 Human oversight points

Point	Who decides	What the system does
Risk class	Client answers; scorer applies fixed rules	Shows the class, the scores behind it and "Answer again"
Investment choices	Client	Greys out options the class does not allow, with the reason
Connecting brokerage data	Client, with explicit versioned consent	Uses read-only data; offers deletion at any time

Accepting a portfolio	Client	Shows rejected candidates and why; labels results hypothetical
Planned trade	Client	Gives a deterministic verdict and explanation; never places the order
Re-checking the profile after a market fall	Client	Offers the re-check; never changes the profile by itself

10.2 Monitoring

Guardrail telemetry: Every compliance-guard block, grounding failure and redaction event is logged as a count, with no message text, so a spike signals an attack or a model regression without exposing client data.

Model regression: The chat and trade-explanation red-team suites (24 cases each) are re-run whenever a prompt or model changes; the Agent D tone evaluation is re-run against the 120 labelled headlines.

Data quality: Data banners tell the client when a fallback source or snapshot was used; the report names its data sources.

Audit trail: Signed-in What-If runs record the scenario, seed, path count, engine version and a fingerprint of the base context, so any result can be reproduced.

10.3 Incident response

Incident	Immediate action (kill switch)	Follow-up
LLM gives harmful or wrong advice	Remove OPENAI_API_KEY on the backend: chat returns 503 and trade explanations fall back to the fixed template; reports and verdicts keep working	Add the case to the red-team suite; adjust guard or prompt; re-run suites
Leaked secret (API key, HMAC secret)	Rotate the key in the host environment; a new EGOQUANT_CONTEXT_SECRET invalidates every sealed context at once	Scan history; record in the risk list

Suspected cross-user data exposure	Disable account routes; revoke Supabase sessions	Run rls_isolation.sql; notify affected users under PIPL
Bad market data	Agent C falls back to the committed real-price snapshot automatically; for a demo, EGOQUANT_DATA=synthetic switches to labelled synthetic data	Compare against a second source; invalidate caches
Regime model misbehaves	Set EGOQUANT_REGIME_MC=0 (falls back to the block bootstrap)	Re-run the walk-forward evaluation
Denial of service or cost spike	Tighten rate limits; scale down What-If workers	Move limits to a shared store (R-06)

Declaration

This self-assessment was prepared by the EgoQuant team from the source code, tests and evaluation logs listed above. Every control claim names the file or test that supports it; items not yet verified in the live environment are marked Partly met or listed in Section 9. EgoQuant is a competition prototype: it is not a licensed investment advisory service, uses no real client money and places no orders.

Sources

1. NFRA Guidance on AI in banking and insurance (Xinhua, 18 Jun 2026) and The Paper summary
2. JR/T 0221-2021 summary (China E-Banking Net) and standard text (PDF)
3. JR/T 0171-2020 (Han Kun)
4. GB/T 45654-2025, English translation (CSET)
5. Measures for Labelling AI-Generated Synthetic Content (CAC)
6. Interim Measures for Generative AI Services (Morrison Foerster)
7. Financial agent standard report (Beijing FinTech Alliance)
8. AMAC fund-distributor suitability guideline (China Securities Journal)
9. Draft group standard on AI algorithm ethics and security in finance
10. CFCA on AI agents in banking

11. OWASP Top 10 for Agentic Applications
12. OWASP Top 10 for LLM Applications 2025 (Invicti)
13. NIST AI 600-1
14. SEC robo-adviser guidance 2017 (Dechert)